

Space Robotics for Wireless Power Transfer: Satellite Constellation Design, Optimization, and Optimal Routing

Raj Priyadarshi
Department of MAE
New York University
Email: rp4206@nyu.edu

Abstract—This project report presents a complete pipeline for designing and analyzing a satellite constellation that beams power to ground rectennas in a space robotics setting. First, the Earth’s surface is partitioned and rectennas are placed using Voronoi-based segmentation on the sphere. Then, rectenna motion (due to Earth’s rotation) and satellite trajectories (inclined circular orbits) are represented in a common 3D coordinate frame. An angle-based activation condition is defined between rectenna and satellite rays, leading to a sampled coverage function $\Phi(\theta_{\text{incl}}, \Omega)$ that depends on orbital inclination and right ascension of ascending node (RAAN). This function is evaluated over a grid of inclination angles from 0° to 90° and RAAN from 0° to 180° with 100 samples in each dimension, yielding 10 000 orbit configurations. Using these samples, we formulate a mixed-integer linear programming (MILP) problem to select a minimal subset of satellites such that every rectenna is active for at least 85% of the time. The resulting solution selects 1183 satellites with distinct $(\theta_{\text{incl}}, \Omega)$ pairs. Finally, we run a high-resolution simulation and construct multi-hop energy transfer paths using graph theory, implemented with a CUDA-accelerated segment feasibility check and a BFS-based path extractor. We discuss the numerical implementation using Python, GPU acceleration, and graph-based routing.

Index Terms—space robotics, wireless power transfer, satellite constellations, MILP, orbital mechanics, graph theory

I. INTRODUCTION

Space robotics increasingly relies on persistent access to energy, whether for in-orbit servicing, surface robots, or distributed sensor networks. Beaming power from satellites to ground-based rectennas is a promising approach to address this challenge, but requires careful constellation design and analysis.

In this work, we consider the following pipeline:

- 1) Partition the Earth and determine rectenna locations using a Voronoi-based segmentation on the sphere.
- 2) Model the motion of rectennas (due to Earth’s rotation) and satellites (circular inclined orbits) in a common 3D inertial frame.
- 3) Define an angle-based activation condition between a satellite and a rectenna, and compute the fraction of time this condition holds.
- 4) Define a coverage function $\Phi(\theta_{\text{incl}}, \Omega)$ that maps an orbit’s inclination and RAAN to the activation probability for each rectenna.
- 5) Sample Φ over a grid: $\theta_{\text{incl}} \in [0^\circ, 90^\circ]$ and $\Omega \in [0^\circ, 180^\circ]$ with 100 samples in each dimension (10 000 orbits).

- 6) Formulate and solve a MILP to select a minimum number of satellites such that each rectenna is active for at least 85% of the time.
- 7) Run a high-resolution short-term simulation and use graph theory to construct multi-hop paths from a source to the rectennas.

The pipeline is implemented using Python with NumPy, PyTorch, PuLP, NetworkX, and related tools. While the focus here is methodological rather than on specific hardware, the framework is directly applicable to constellation design problems in space robotics.

II. EARTH AND RECTENNA GEOMETRY

A. Earth Model

We model the Earth as a perfect sphere of radius $R_E \approx 6371$ km. Using an Earth-centered inertial (ECI) frame, a point on the surface at latitude φ and longitude λ (in radians) is mapped into 3D as

$$\mathbf{p}(\varphi, \lambda) = R_E \begin{bmatrix} \cos \varphi \cos \lambda \\ \cos \varphi \sin \lambda \\ \sin \varphi \end{bmatrix} \in \mathbb{R}^3. \quad (1)$$

B. Initial Partitioning and Voronoi-Based Rectenna Placement

We begin with an approximate equal-area partitioning of the Earth’s surface. Let

$$\{(\varphi_i, \lambda_i)\}_{i=1}^{N_{\text{cand}}} \quad (2)$$

denote an initial set of candidate points in latitude–longitude coordinates, with associated 3D positions

$$\mathbf{c}_i = \mathbf{p}(\varphi_i, \lambda_i). \quad (3)$$

We then perform a Voronoi-based segmentation on the sphere: the surface is partitioned into Voronoi cells $\{V_j\}$ induced by the candidate positions. For each cell V_j , we approximate a centroid direction by averaging the normalized points:

$$\mathbf{m}_j = \frac{1}{|V_j|} \sum_{\mathbf{x} \in V_j} \frac{\mathbf{x}}{\|\mathbf{x}\|}, \quad \hat{\mathbf{m}}_j = \frac{\mathbf{m}_j}{\|\mathbf{m}_j\|}. \quad (4)$$

The final rectenna position for cell j is then

$$\mathbf{r}_j = R_E \hat{\mathbf{m}}_j, \quad j = 1, \dots, N_{\text{rect}}. \quad (5)$$



Fig. 1: Map of Earth.



Fig. 2: Voronoi segmented map of Earth.

III. RECTENNA MOTION IN 3D

The Earth rotates with angular velocity

$$\omega_E = \frac{2\pi}{T_E}, \quad T_E = 86\,400 \text{ s}. \quad (6)$$

For rectenna r with fixed latitude φ_r and initial longitude λ_r^0 at $t = 0$, the longitude in the inertial frame at time t is

$$\lambda_r(t) = \lambda_r^0 + \omega_E t. \quad (7)$$

Hence the rectenna's position is

$$\mathbf{p}_{\text{rect},r}(t) = R_E \begin{bmatrix} \cos \varphi_r \cos(\lambda_r^0 + \omega_E t) \\ \cos \varphi_r \sin(\lambda_r^0 + \omega_E t) \\ \sin \varphi_r \end{bmatrix}. \quad (8)$$

Its unit direction vector in the ECI frame is

$$\hat{\mathbf{e}}_{\text{rect},r}(t) = \frac{\mathbf{p}_{\text{rect},r}(t)}{\|\mathbf{p}_{\text{rect},r}(t)\|} = \frac{\mathbf{p}_{\text{rect},r}(t)}{R_E}. \quad (9)$$

IV. SATELLITE ORBITAL MODEL

Each satellite is assumed to be in a circular orbit of radius

$$r_{\text{orbit}} = R_E + h, \quad h \approx 500 \text{ km}, \quad (10)$$

specified by its inclination θ_{incl} and RAAN Ω .

A. Orbital Plane Basis

Let $\mathbf{I} = [1, 0, 0]^T$, $\mathbf{J} = [0, 1, 0]^T$ and $\mathbf{K} = [0, 0, 1]^T$. The line of nodes has direction

$$\mathbf{u} = \cos \Omega \mathbf{I} + \sin \Omega \mathbf{J}. \quad (11)$$

A vector perpendicular to $\mathbf{u}$ in the equatorial plane is

$$\mathbf{f} = \mathbf{K} \times \mathbf{u}. \quad (12)$$

We rotate $\mathbf{f}$ about $\mathbf{u}$ by θ_{incl} using Rodrigues' formula:

$$\mathbf{v} = \mathbf{f} \cos \theta_{\text{incl}} + (\mathbf{u} \times \mathbf{f}) \sin \theta_{\text{incl}} + \mathbf{u}(\mathbf{u} \cdot \mathbf{f})(1 - \cos \theta_{\text{incl}}), \quad (13)$$

and the pair $(\mathbf{u}, \mathbf{v})$ spans the orbital plane.

B. Satellite Position and Angular Velocity

Let $\mu = GM_E$ be the standard gravitational parameter of Earth. The satellite's angular velocity for a circular orbit is

$$\omega_{\text{sat}} = \sqrt{\frac{\mu}{r_{\text{orbit}}^3}}. \quad (14)$$

Let ϕ_0 be an initial phase. Then the satellite position is

$$\mathbf{p}_{\text{sat}}(t) = r_{\text{orbit}} [\cos(\omega_{\text{sat}} t + \phi_0) \mathbf{u} + \sin(\omega_{\text{sat}} t + \phi_0) \mathbf{v}], \quad (15)$$

with unit vector

$$\hat{\mathbf{e}}_{\text{sat}}(t) = \frac{\mathbf{p}_{\text{sat}}(t)}{\|\mathbf{p}_{\text{sat}}(t)\|} = \frac{\mathbf{p}_{\text{sat}}(t)}{r_{\text{orbit}}}. \quad (16)$$

V. ANGLE-BASED ACTIVATION & COVERAGE

A. Activation Condition

For a satellite s and rectenna r , define the angle $\alpha_{s,r}(t)$ via the dot product:

$$\cos \alpha_{s,r}(t) = \hat{\mathbf{e}}_{\text{sat},s}(t) \cdot \hat{\mathbf{e}}_{\text{rect},r}(t), \quad (17)$$

$$\alpha_{s,r}(t) = \arccos(\hat{\mathbf{e}}_{\text{sat},s}(t) \cdot \hat{\mathbf{e}}_{\text{rect},r}(t)). \quad (18)$$

We additionally require that the vectors point in the same general direction, i.e.,

$$\mathbf{p}_{\text{sat},s}(t) \cdot \mathbf{p}_{\text{rect},r}(t) > 0. \quad (19)$$

Let α_{th} be the angle threshold (e.g. $\alpha_{\text{th}} = 3^\circ$). We define the activation indicator

$$I_{s,r}(t) = \begin{cases} 1, & \alpha_{s,r}(t) \leq \alpha_{\text{th}} \text{ and } \mathbf{p}_{\text{sat},s}(t) \cdot \mathbf{p}_{\text{rect},r}(t) > 0, \\ 0, & \text{otherwise.} \end{cases} \quad (20)$$

B. Time-Averaged Activation Probability

Over a time horizon $[0, T_{\text{hor}}]$ (here, $T_{\text{hor}} = 5$ days), the activation probability is

$$P_{s,r} = \frac{1}{T_{\text{hor}}} \int_0^{T_{\text{hor}}} I_{s,r}(t) dt. \quad (21)$$

We approximate this via sampling with period Δt :

$$t_n = n\Delta t, \quad n = 0, \dots, N_T - 1, \quad N_T = \left\lfloor \frac{T_{\text{hor}}}{\Delta t} \right\rfloor, \quad (22)$$

yielding the estimator

$$\hat{P}_{s,r} \approx \frac{1}{N_T} \sum_{n=0}^{N_T-1} I_{s,r}(t_n). \quad (23)$$

VI. COVERAGE FUNCTION $\Phi(\theta_{\text{incl}}, \Omega)$

For each rectenna r we define

$$\Phi_r(\theta_{\text{incl}}, \Omega) \triangleq \text{fraction of time rectenna } r \text{ is active for given } (\theta_{\text{incl}}, \Omega). \quad (24)$$

Using the discrete estimator,

$$\Phi_r(\theta_{\text{incl}}, \Omega) \approx \hat{P}_{s(\theta_{\text{incl}}, \Omega), r} = \frac{1}{N_T} \sum_{n=0}^{N_T-1} I_{s(\theta_{\text{incl}}, \Omega), r}(t_n), \quad (25)$$

where $s(\theta_{\text{incl}}, \Omega)$ denotes the satellite orbit with these parameters.

Analytically characterizing Φ_r in closed form is intractable due to the coupled rotations of Earth and orbit dynamics. We therefore evaluate it numerically.

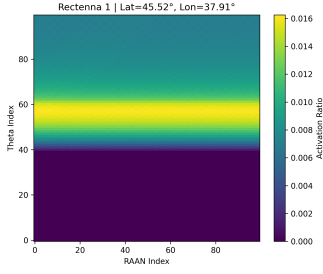


Fig. 3: Coverage function for a rectenna placed at latitude 45.52° and longitude 37.91°.

A. Sampling Grid

We consider

$$\theta_{\text{incl}} \in \{\theta_1, \dots, \theta_{N_\theta}\}, \quad \theta_1 = 0^\circ, \theta_{N_\theta} = 90^\circ, N_\theta = 100, \quad (26)$$

$$\Omega \in \{\Omega_1, \dots, \Omega_{N_\Omega}\}, \quad \Omega_1 = 0^\circ, \Omega_{N_\Omega} = 180^\circ, N_\Omega = 100. \quad (27)$$

This yields $S = N_\theta N_\Omega = 10\,000$ orbit configurations, indexed by $s = 1, \dots, S$. For each satellite configuration s and rectenna r , we store

$$\Phi_{s,r} \in [0, 1], \quad (28)$$

resulting in a coverage matrix

$$\Phi \in \mathbb{R}^{S \times N_{\text{rect}}}. \quad (29)$$

VII. MILP FOR CONSTELLATION DESIGN

We now aim to select a minimal set of satellites such that each rectenna is active at least 85% of the time.

A. Decision Variables

For each satellite configuration $s \in \{1, \dots, S\}$, introduce a binary variable

$$x_s = \begin{cases} 1, & \text{if configuration } s \text{ is chosen,} \\ 0, & \text{otherwise.} \end{cases} \quad (30)$$

Let $\mathbf{x} = [x_1, \dots, x_S]^T$ and

$$\alpha_{\text{target}} = 0.85. \quad (31)$$

B. Coverage Constraints

For each rectenna r , the total coverage must satisfy

$$\sum_{s=1}^S \Phi_{s,r} x_s \geq \alpha_{\text{target}}. \quad (32)$$

In matrix form, letting $\Phi \in \mathbb{R}^{S \times N_{\text{rect}}}$ and $\mathbf{1}_R$ be the all-ones vector in $\mathbb{R}^{N_{\text{rect}}}$:

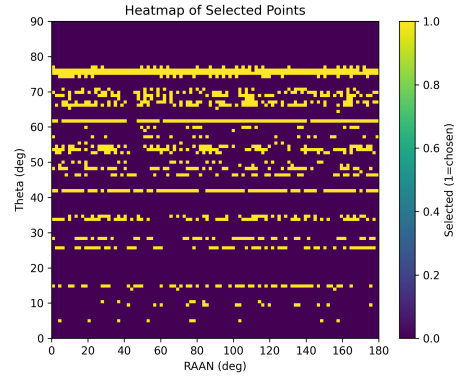
$$\Phi^T \mathbf{x} \geq \alpha_{\text{target}} \mathbf{1}_R. \quad (33)$$

C. Objective

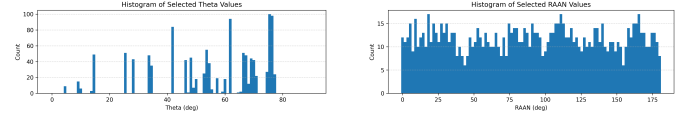
We minimize the number of selected satellites:

$$\min_{\mathbf{x} \in \{0,1\}^S} \mathbf{1}_S^T \mathbf{x}, \quad (34)$$

where $\mathbf{1}_S$ is the S -dimensional vector of ones.



(a) Inclination θ and RAAN for selected satellites.



(b) Histogram of inclination angles θ . (c) Histogram of RAAN angles Ω .

Fig. 4: Selected satellite orbits in the $(\theta_{\text{incl}}, \Omega)$ plane and the corresponding marginal distributions.

D. MILP Summary

The full MILP is:

$$\min_{\mathbf{x}} \mathbf{1}_S^T \mathbf{x} \quad (35)$$

$$\text{s.t. } \Phi^T \mathbf{x} \geq \alpha_{\text{target}} \mathbf{1}_R, \quad (36)$$

$$\mathbf{x} \in \{0, 1\}^S. \quad (37)$$

We solve this using PuLP with the CBC solver. In the reported experiments, the optimal (or near-optimal) solution selects

$$S_{\text{chosen}} = 1183 \quad (38)$$

orbits, each corresponding to a unique pair $(\theta_{\text{incl}}, \Omega)$.

VIII. HIGH-RESOLUTION SIMULATION AND MULTI-HOP ROUTING

A. Short-Term High-Resolution Simulation

To visualize and further analyze the constellation, we simulate a shorter horizon

$$T_{\text{sim}} = 2.4 \text{ hours} = 2.4 \times 3600 \text{ s}, \quad \Delta t_{\text{sim}} = 10 \text{ s}, \quad (39)$$

with number of simulation steps

$$N_{\text{sim}} = \left\lceil \frac{T_{\text{sim}}}{\Delta t_{\text{sim}}} \right\rceil. \quad (40)$$

At each $t_n = n\Delta t_{\text{sim}}$, we compute

$$\mathbf{p}_{\text{sat},s}(t_n) \in \mathbb{R}^3, \quad (41)$$

$$\mathbf{p}_{\text{rect},r}(t_n) \in \mathbb{R}^3, \quad (42)$$

and re-evaluate the activation indicators $I_{s,r}(t_n)$, which specify which rectennas are illuminated by which satellites at that time.

B. CUDA-Accelerated Multihop Path Solver

To build multi-hop paths from a source point to all rectennas over all time steps, we implement a CUDA-

accelerated algorithm that mirrors the Python function `compute_all_paths_from_source_multihop_no_earth_level`.

1) *Data Layout and Node Indexing*: Let N_{sat} be the number of satellites, N_{rect} the number of rectennas, and T the number of simulation time steps. The input arrays are

$$\text{sat_positions_m} \in \mathbb{R}^{N_{\text{sat}} \times T \times 3}, \quad (43)$$

$$\text{rect_positions_m} \in \mathbb{R}^{N_{\text{rect}} \times T \times 3}, \quad (44)$$

$$\text{visibility_mask} \in \{0, 1\}^{N_{\text{sat}} \times N_{\text{rect}} \times T}. \quad (45)$$

We also have a fixed source position $\mathbf{p}_S \in \mathbb{R}^3$.

For each time step we construct a graph with

$$N_{\text{nodes}} = 1 + N_{\text{sat}} + N_{\text{rect}} \quad (46)$$

nodes: index 0 is the source S , indices $1 + i$ correspond to satellite i ($i = 0, \dots, N_{\text{sat}} - 1$), and indices $1 + N_{\text{sat}} + r$ to rectenna r ($r = 0, \dots, N_{\text{rect}} - 1$).

2) *Vectorized Segment Feasibility Kernel*: On the GPU we evaluate, in batch, the feasibility of segments between pairs of points. Given batched endpoints $\mathbf{P}_1, \mathbf{P}_2 \in \mathbb{R}^{B \times 3}$, we compute

$$\mathbf{d} = \mathbf{P}_2 - \mathbf{P}_1, \quad \mathbf{f} = \mathbf{P}_1, \quad (47)$$

and scalars

$$a = \mathbf{d} \cdot \mathbf{d}, \quad b = 2(\mathbf{f} \cdot \mathbf{d}), \quad c = \mathbf{f} \cdot \mathbf{f} - R^2, \quad (48)$$

with discriminant

$$\Delta = b^2 - 4ac. \quad (49)$$

Using a fully vectorized implementation in PyTorch, we compute

$$t_{1,2} = \frac{-b \pm \sqrt{\max(\Delta, 0)}}{2a}, \quad (50)$$

with appropriate masking for degenerate segments where $a \approx 0$. A Boolean feasibility tensor (no intersection in the forbidden region) is obtained by checking whether the parameters $t_{1,2}$ lie outside the segment interval $[0, 1]$.

3) *GPU-Based Adjacency Computation*: For each time step t we perform the following operations on the GPU:

1) Extract current positions:

$$\mathbf{P}_{\text{sat}}(t) = \text{sat_positions_m}[:, t, :], \quad (51)$$

$$\mathbf{P}_{\text{rect}}(t) = \text{rect_positions_m}[:, t, :]. \quad (52)$$

2) Stack segments from source to all satellites and evaluate feasibility, producing a Boolean mask $\text{ok}_{S\text{-sat}} \in \{0, 1\}^{N_{\text{sat}}}$.

3) Stack all satellite-satellite pairs to obtain $\text{ok}_{\text{sat-sat}} \in \{0, 1\}^{N_{\text{sat}} \times N_{\text{sat}}}$.

4) Stack satellite-rectenna pairs into a tensor of shape $N_{\text{sat}} \times N_{\text{rect}} \times 3$ and combine geometric feasibility with the visibility mask:

$$\text{ok}_{\text{sat-rect}} = \text{ok}_{\text{geo}} \wedge \text{visibility_mask}(:, :, t). \quad (53)$$

4) *CPU Graph Assembly and BFS*: After computing the feasibility masks, we transfer them back to the CPU and assemble adjacency lists for an undirected graph $G_t = (V, E_t)$:

- For each satellite i with $\text{ok}_{S\text{-sat}}[i] = 1$, we add edges (S, sat_i) and (sat_i, S) .
- For each pair of satellites (i, j) with $\text{ok}_{\text{sat-sat}}[i, j] = 1$, we add edges $(\text{sat}_i, \text{sat}_j)$ and $(\text{sat}_j, \text{sat}_i)$.

- For each satellite-rectenna pair (i, r) with $\text{ok}_{\text{sat-rect}}[i, r] = 1$, we add edges $(\text{sat}_i, \text{rect}_r)$ and $(\text{rect}_r, \text{sat}_i)$.

We then run a breadth-first search (BFS) from the source node S , storing a parent array $\text{parent}[v]$ for all $v \in V$. If $\text{parent}[v] = -1$, the node v is unreachable at that time step.

5) *Path Reconstruction and Storage*: For each rectenna index r with node index v_r , we reconstruct a path if reachable:

$$S \rightarrow \text{sat}_{i_1} \rightarrow \dots \rightarrow \text{sat}_{i_k} \rightarrow \text{rect}_r, \quad (54)$$

by following the parent pointers backwards from v_r to the source node and reversing the resulting sequence of node indices. Using the node positions at time t , we form the 3D path

$$[\mathbf{p}_S, \mathbf{p}_{\text{sat}, i_1}(t), \dots, \mathbf{p}_{\text{sat}, i_k}(t), \mathbf{p}_{\text{rect}, r}(t)] \in \mathbb{R}^{(k+2) \times 3}, \quad (55)$$

and store it in

$$\text{paths_matrix}[r][t]. \quad (56)$$

If no path exists, we set $\text{paths_matrix}[r][t] = \text{Null}$. Overall, the algorithm produces a list-of-lists data structure of shape $N_{\text{rect}} \times T$ containing either feasible multi-hop paths or `Null` entries.

C. Graph Construction and Path Extraction

The combination of GPU-accelerated feasibility checks and CPU BFS results in an efficient multi-hop routing scheme:

- 1) The cost of geometric feasibility and visibility checks, which dominates for large constellations, is moved to the GPU and vectorized.
- 2) The BFS on each G_t has complexity $\mathcal{O}(|V| + |E|)$ and runs quickly on the CPU since the graphs are sparse and of moderate size.
- 3) The resulting `paths_matrix` can be post-processed to compute statistics such as the number of rectennas with valid paths per time step, the length distribution of multi-hop chains, and the aggregate availability of energy routes.

IX. IMPLEMENTATION AND TOOLS

The complete pipeline is implemented in Python.

- **NumPy / Pandas**: data management, vectorized math.
- **PyTorch**: large-scale simulation and vectorized operations on the GPU.
- **PuLP + CBC**: MILP modeling and solving.
- **NetworkX**: graph construction and multi-hop path search.
- **tqdm**: progress bars for long-running computations.

Simulation of the 5-day horizon and evaluation of $\Phi_{s,r}$ over 10 000 orbit configurations is accelerated using the GPU via PyTorch. The MILP is solved in a separate step, and the resulting subset of 1183 satellites is used for the high-resolution simulation and graph-based routing.

X. CONCLUSION

We presented a complete workflow for designing and analyzing a satellite constellation for wireless power transfer in a space robotics context. Starting from an Earth partitioning and Voronoi-based rectenna placement, we derived 3D motion

models for rectennas and satellites, defined an angle-based activation condition, and constructed a coverage function $\Phi(\theta_{\text{incl}}, \Omega)$. By sampling this function over a 10 000-point orbit grid and formulating a MILP, we obtained a constellation of 1183 satellites such that all rectennas are active at least 85% of the time. Finally, we demonstrated a high-resolution simulation and multi-hop routing algorithm implemented with CUDA-accelerated feasibility checks and BFS-based path extraction.

Future work includes joint optimization of rectenna and satellite placements, incorporation of link budgets and power

constraints, handling failures and degraded satellites, and extending the orbital models to include perturbations and eccentric orbits.

REFERENCES

- [1] D. Bertsekas, *Convex Optimization Theory*. Athena Scientific, 2009.
- [2] D. A. Vallado, *Fundamentals of Astrodynamics and Applications*. Springer, 2001.
- [3] S. Boyd and L. Vandenberghe, *Convex Optimization*. Cambridge University Press, 2004.